

TÀI LIỆU HƯỚNG DẪN VIBE CODING

Claude Code Playbook

Quy trình làm việc với AI cho team TalentChart

1 mid dev + 2-3 sinh viên CNTT HPU

Tài liệu này dạy team dùng Claude Code + Claude.ai chat hiệu quả để:

- Tận dụng meta-architecture HPU + 7 skills + 7 slash commands
 - Sinh code chất lượng cao gấp 3-5 lần dev tay
 - Tránh các lỗi bảo mật phổ biến do AI sinh ra
 - Multi-tenant safety bằng pattern auto-enforced

Đọc TRƯỚC khi bắt đầu Sprint 1

Phiên bản 1.0 · Tháng 4 năm 2026 · HPU & SCEI

MỤC LỤC

1. Triết lý vibe coding với meta-architecture HPU.....	3
2. Setup Claude Code + 2 workflow chuẩn.....	5
3. Quy trình làm việc 6 bước cho mỗi task.....	9
4. Cách dùng 7 skills hiệu quả.....	12
5. Cách dùng 7 slash commands.....	16
6. Bộ prompt templates đã tối ưu.....	19
7. Code review checklist (BẮT BUỘC).....	27
8. 12 anti-patterns AI hay sinh ra.....	30
9. Workflow Git và CI/CD.....	34
10. Khi nào KHÔNG dùng AI.....	37
11. Phân chia công việc cho team 1+2+1.....	39
12. Đo lường năng suất + cải tiến.....	42

1. TRIẾT LÝ VIBE CODING VỚI META-ARCHITECTURE HPU

Nguyên tắc cốt lõi

Claude là junior dev cực thông minh, đọc nhanh, biết nhiều framework, nhưng KHÔNG biết:

- Convention của HPU (anh đã có sẵn trong `_shared/`)
- Skills + slash commands của HPU (anh đã setup `~/.claude/skills/`)
- Bối cảnh dự án TalentChart (CLAUDE.md kể)
- Quyết định kiến trúc của team (DECISIONS.md kể)

Bộ hpu-dev v2.0 chính là cách ANH dạy Claude biết những điều trên — một lần, dùng cho mọi project.

Vibe coding là gì và không là gì?

Vibe coding LÀ:

- Mô tả ý tưởng bằng ngôn ngữ tự nhiên (kèm context cụ thể), để AI sinh code
- Đọc code AI sinh ra một cách kỹ lưỡng, hiểu từng dòng
- Sửa, từ chối, hoặc yêu cầu sinh lại nếu code không đúng ý
- Giảm thời gian gõ code thuần túy từ 70% xuống 20% workday
- Dành 80% thời gian còn lại cho thiết kế, review, test, debug

Vibe coding KHÔNG phải là:

- Copy-paste mù vào project mà không đọc — đây là tự sát kỹ thuật
- Tin tuyệt đối vào AI mà không verify
- Dùng AI thay thế việc hiểu code — khi sản phẩm có bug production, anh phải debug, AI không có ở đó
- Gửi 1 prompt khổng lồ để AI làm cả feature lớn — chia nhỏ luôn cho kết quả tốt hơn
- Lý do để skip testing — code AI sinh càng cần test nhiều hơn code dev tay

Tại sao bộ hpu-dev v2.0 quan trọng?

Trước khi có hpu-dev, mỗi conversation với Claude là 1 lần kể lại context từ đầu:

- "Project tôi dùng FastAPI + PostgreSQL..." (lặp đi lặp lại)
- "API trả envelope status/data/message..." (lặp)
- "Dùng soft delete với status=inactive..." (lặp)

- "Multi-tenant với organization_id..." (lặp)

Sau khi có hpu-dev:

- Convention nằm trong _shared/, đọc 1 lần
- Skills auto-load qua trigger keywords (Claude Code đọc YAML metadata)
- Slash commands viết sẵn cho task lặp đi lặp lại (/new-module, /security-review...)
- Auto-rules trong .claude/rules/ — Claude tự kiểm tra trước khi sinh code

Effort tiết kiệm thực tế

Không có hpu-dev: mỗi task tốn 5-10 phút giải thích context + 30 phút sửa code AI sinh ra theo convention.

Có hpu-dev: context có sẵn, code đầu ra đã đúng convention 90%, chỉ cần 5 phút review.

Tính cho 1 mid + 2 intern × 4 tháng Phase 1: tiết kiệm ~200 giờ làm việc.

4 mức trưởng thành của dev khi dùng Claude Code

Roadmap năng lực — team phấn đấu lên Level 3-4

Level 1 — Thợ copy-paste:

Hỏi Claude → copy code → paste → chạy thử. Không hiểu code, không edit. Code chạy thì OK, không chạy thì hỏi lại 'sửa giúp tôi'. NGUY HIỂM: hay đẻ ra bug security và logic.

Level 2 — Thợ verify:

Đọc code Claude sinh, hiểu 70%, copy phần dùng được. Biết chạy test cơ bản. Đa số intern bắt đầu ở đây.

Level 3 — Pair programmer:

Trao đổi 2 chiều với Claude, đẩy lùi khi code không tốt, yêu cầu cải tiến. Đọc hiểu 100% code trước khi merge. Mid dev nên ở mức này.

Level 4 — Architect:

Dùng Claude để hiện thực hóa thiết kế của mình, không bị Claude dẫn dắt. Biết khi nào KHÔNG dùng AI. Tự viết skills + slash commands cho nhu cầu đặc thù. Trung ở mức này.

Nguyên tắc 'Trust but verify'

AI có thể sinh code:

- Hoạt động đúng 80% trường hợp, sai ở edge case
- Có lỗ hổng bảo mật mà chính AI không nhận ra (SQL injection trong raw query)
- Trông đẹp nhưng không phù hợp với codebase của mình
- Dùng API/library bị deprecated
- Mock data hoặc placeholder mà AI quên thay (TODO, FIXME, your-key-here)

Trust để dám dùng AI tăng tốc 5x. Verify để không để ra bug bom hẹn giờ. 2 vế song hành.

2. SETUP CLAUDE CODE + 2 WORKFLOW CHUẨN

2.1. Cài đặt Claude Code

Claude Code là CLI tool của Anthropic, chạy trong terminal, đọc CLAUDE.md tự động và load skills theo trigger keywords. Đây là tool CHÍNH cho team TalentChart.

Cài đặt

```
# Cài Node.js 18+ trước (nếu chưa có)
# Trên Ubuntu/Debian:
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs

# Cài Claude Code
npm install -g @anthropic-ai/claude-code

# Verify
claude --version

# Login lần đầu (sẽ mở browser)
claude

# Follow prompt để authenticate với Anthropic account
```

Setup hpu-dev ecosystem

```
# 1. Giải nén bộ hpu-dev v2.0
cd ~/Downloads
unzip hpu-dev-v2.zip
mv hpu-dev ~/

# 2. Chạy setup (cài skills + global CLAUDE.md + Docker network)
cd ~/hpu-dev
chmod +x setup.sh
./setup.sh

# Verify skills đã cài
ls ~/.claude/skills/
# Expect: docker-deploy epa-engine fastapi-crud hpu-ui multi-tenant-saas
#           nextjs-admin vietnam-payroll

# Verify global CLAUDE.md
cat ~/.claude/CLAUDE.md
```

2.2. Tạo project TalentChart

```
# Tạo project SaaS TalentChart
~/hpu-dev/new-project.sh talentchart "Nền tảng SaaS tuyển dụng + EPA Engine" 8003
--type=saas

# Vào project
cd ~/hpu-dev/talentchart

# Setup .env từ template
cp .env.example .env
nano .env # Điền giá trị (xem hướng dẫn ở chương 9)

# Mở Claude Code
claude
```

Claude Code sẽ TỰ ĐỘNG:

- Đọc ~/.claude/CLAUDE.md (global preferences của Trung)
- Đọc ./CLAUDE.md (project-specific cho TalentChart)
- Phát hiện đây là SaaS qua CLAUDE.md → load skills phù hợp
- Đọc .claude/rules/ — biết các luật cấm (raw SQL, hard delete, ...)
- Biết các slash commands available (/new-module, /new-tenant, /epa-port...)

Lần đầu mở Claude Code trên project mới, hỏi câu này:

"Đọc CLAUDE.md, _shared/api-conventions.md, _shared/db-conventions.md.

Sau đó tóm tắt: project này làm gì, stack gì, conventions chính, multi-tenant pattern, slash commands available."

Claude sẽ tóm tắt context — anh verify Claude đã 'hiểu' đúng project trước khi giao task.

2.3. Hai workflow chuẩn

Workflow A — Claude Code làm chính (cho mid dev)

Áp dụng cho: mid dev khi build feature lớn, refactor, hoặc cần Claude tự chạy lệnh (test, lint, git):

```
# Mở terminal trong thư mục project
cd ~/hpu-dev/talentchart
claude
```

```
# Trong Claude Code, gõ task:
> Đọc docs/PLAN.md sprint hiện tại. Implement story BE-007:
> "Build endpoint POST /api/v1/candidates/{id}/move với pipeline 7 trạng thái"

# Claude sẽ:
# 1. Đọc PLAN.md để hiểu story
# 2. Đọc DECISIONS.md để biết quyết định liên quan
# 3. Load skill fastapi-crud + multi-tenant-saas
# 4. Đọc các file liên quan trong codebase
# 5. Đề xuất plan
# 6. Sau khi anh confirm: viết code + tests
# 7. Chạy pytest tự động
# 8. Báo cáo kết quả
```

Lợi điểm Workflow A:

- Claude truy cập được file system → đọc/sửa nhiều file trong 1 task
- Claude chạy được lệnh shell → run test, migration, lint tự động
- Context tự động → không cần paste file
- Slash commands hoạt động (/new-module, /security-review...)
- Skills auto-load theo trigger words

Workflow B — Claude.ai chat làm phụ (cho intern + Trung review)

Áp dụng cho: brainstorm trước khi code, review code Claude Code đã sinh ra, hoặc khi không có terminal:

Khi nào dùng Workflow B:

1. Brainstorm thiết kế trước khi code (Trung làm với Claude.ai trên web)
2. Output là spec → paste vào docs/PLAN.md của project
3. Mid dev mở Claude Code → 'Đọc docs/PLAN.md và implement'
4. Sau khi code xong, Trung review qua Claude.ai (paste code vào)
5. Feedback → mid dev sửa với Claude Code

Đây là workflow 2 lớp: Claude.ai (think) + Claude Code (do).

2.4. Project Knowledge cho Claude.ai (cho Workflow B)

Trên Claude.ai (web), tạo 1 Project chung 'TalentChart' để brainstorm và review:

- Vào claude.ai → Projects → New Project

- Tên: 'TalentChart'
- Description: copy đoạn từ CLAUDE.md của project

System prompt cho Project

PROMPT — System prompt cho Project TalentChart trên Claude.ai

Bạn đang giúp team build TalentChart – nền tảng SaaS B2B tuyển dụng và đánh giá nhân sự cho trường ĐH VN, có module Eastern Personality Assessment (bát tự + 12 archetype).

Project tuân thủ HPU meta-architecture v2.0. Conventions chính:

- Backend: FastAPI 0.115+ + Python 3.12 + SQLAlchemy 2.0 async + PostgreSQL 16 + Pydantic v2 + Alembic
- Frontend: Next.js 15 App Router + TypeScript 5.6 + Tailwind 3.4 + shadcn/ui
- API envelope: {status, data, message} qua helper success()/error()/paginated()
- Multi-tenant: mọi bảng tenant-scoped có organization_id NOT NULL + RLS
- Soft delete: status="inactive", NEVER hard delete
- Money: Integer VNĐ
- Cross-tenant access trả 404 (không 403)
- Pydantic Create schema KHÔNG có organization_id

VAI TRÒ CHÍNH của bạn trong workflow này:

1. BRAINSTORM thiết kế trước khi code (output: spec dạng markdown)
2. REVIEW code Claude Code đã sinh ra (input: code paste vào)
3. DEBUG khi gặp lỗi (input: error trace)

KHÔNG sinh code dài để copy-paste. Output của bạn là spec/review/feedback, không phải code production.

Reference files cần upload vào Project Knowledge

- TalentChart_Architecture_Phase12.pdf (file 2)
- CLAUDE.md của project (sau khi đã setup)
- _shared/api-conventions.md (cho convention reference)
- _shared/db-conventions.md
- DECISIONS.md (cập nhật khi có quyết định mới)

2.5. Khi nào dùng Workflow nào

Tình huống	Workflow nên dùng
Brainstorm tính năng mới, viết PLAN.md	B (Claude.ai chat)
Implement story trong sprint	A (Claude Code)
Refactor đa file (>5 file)	A (Claude Code)

Tình huống	Workflow nên dùng
Build module mới với /new-module	A (Claude Code)
Port logic JS → Python với /epa-port	A (Claude Code)
Review PR sau khi mid dev xong	B (Claude.ai chat)
Debug lỗi với traceback dài	A (Claude Code có thể đọc log)
Học cách làm task mới	B (Claude.ai chat — giải thích từ tốn)
Security audit toàn module	A (/security-review slash command)
Architecture decision discussion	B (Claude.ai chat — bàn bạc)

Trong Phase 1, ước tính 80% công việc dùng Workflow A (Claude Code), 20% Workflow B (Claude.ai chat).

3. QUY TRÌNH LÀM VIỆC 6 BƯỚC CHO MỖI TASK

3.1. Workflow chuẩn

Mỗi task (Jira ticket, GitHub issue) đi qua 6 bước. Đừng bỏ bước, đừng đảo thứ tự — đó là thứ làm cho vibe coding tạo ra code chất lượng cao thay vì code rác.

WORKFLOW 6 BƯỚC

1. UNDERSTAND — Đọc & hiểu yêu cầu (5-15 phút, KHÔNG mở Claude)
2. PLAN — Brainstorm với Claude.ai chat, output spec (10-20 phút)
3. IMPLEMENT — Mở Claude Code, đọc PLAN, sinh code (15-60 phút)
4. TEST — `/security-review` + `/review` + chạy test (15-30 phút)
5. COMMIT — Conventional commits, update CHANGELOG (5 phút)
6. PR — Tạo PR, request review từ Trung (5 phút)

Tổng: 1-2 giờ cho task vừa, 3-4 giờ cho task lớn

Bước 1 — UNDERSTAND (đừng skip)

Trước khi mở bất kỳ AI tool nào, đọc kỹ yêu cầu. Trả lời tự mình các câu sau:

- Task này thuộc module nào? (Auth/Candidates/Campaigns/EPA/JobPosts...)
- Input: API nhận gì từ client? Schema gì?
- Output: trả gì? Status code nào cho mỗi case?
- Permissions: ai được dùng? (xem `org_role` hierarchy)
- Database: tạo/sửa/xoá/đọc bằng nào? Có cần migration?
- Side effects: gửi email? Tạo audit log? Trigger background job?
- Edge cases: input rỗng? trùng? quá dài? user không có quyền?
- Multi-tenant: có cần verify ownership không? (gần như luôn CÓ)

Nếu không trả lời được câu nào — hỏi Trung trước khi mở AI. Đừng hỏi Claude vì Claude sẽ đoán mà anh chưa biết để verify.

Bước 2 — PLAN (Claude.ai chat, brainstorm trước)

Mở Claude.ai web, conversation mới trong Project TalentChart. Hỏi PLAN trước, KHÔNG hỏi CODE:

PROMPT — Hỏi PLAN trước khi code (Claude.ai chat)

Tôi cần build endpoint `POST /api/v1/candidates/{id}/move` để chuyển candidate sang trạng thái khác trong pipeline.

Trước khi viết code, anh giúp tôi PLAN:

1. Pydantic schema cho request body cần những field gì?
2. Validation logic: chuyển trạng thái nào → trạng thái nào là hợp lệ? (theo pipeline 7 trạng thái: NEW, SCREENING, TEST_SENT, TEST_DONE, INTERVIEW, DECISION, HIRED, REJECTED)
3. Permissions: ai được phép chuyển trạng thái?
4. Side effects: cần ghi audit log không? Gửi email cho candidate không?
5. Multi-tenant: cần verify gì?
6. Error cases và HTTP status code tương ứng?

Đề xuất plan rồi đợi tôi confirm, KHÔNG viết code vội.

Claude sẽ đề xuất plan. Đọc kỹ. Có thể anh phát hiện ra:

- Claude bỏ sót 1 trạng thái không hợp lệ → bổ sung trước khi code
- Claude đề xuất gửi email mà chưa có infra → từ chối, để Phase 2
- Claude muốn thêm field không cần → loại

Khi plan đã chốt, copy plan vào `docs/PLAN.md` của project (hoặc tạo file riêng cho story này), rồi chuyển sang bước 3.

Bước 3 — IMPLEMENT (Claude Code)

Mở terminal, vào project, chạy claude:

```
cd ~/hpu-dev/talentchart
claude

# Trong Claude Code:
> Đọc docs/PLAN.md (story BE-007 vừa được Trung add).
> Implement endpoint POST /api/v1/candidates/{id}/move theo plan đã chốt.
>
> Tuân thủ:
> - skill multi-tenant-saas (tenant verify, RLS)
> - skill fastapi-crud (envelope response)
> - rule .claude/rules/tenant-isolation.md
> - rule .claude/rules/api-response.md
>
> Output: list các file sẽ tạo/sửa, plan implementation.
> Đợi tôi confirm trước khi viết code.
```

Claude Code sẽ:

- Đọc docs/PLAN.md
- Đọc các file model, schema, router liên quan để hiểu pattern hiện tại

- Đề xuất kế hoạch (file nào sẽ tạo/sửa)
- Sau khi anh confirm: thực hiện sinh code
- Tự chạy lint/typecheck
- Báo cáo kết quả + đề xuất tiếp

Bước 4 — TEST (Claude Code tự chạy)

Trong cùng Claude Code session:

```
> Viết tests cho endpoint vừa implement.
> Cover các case:
> - Happy path: HR manager chuyển NEW → SCREENING (200)
> - Invalid transition: chuyển NEW → HIRED (400)
> - Permission denied: viewer (403)
> - Not found: candidate id không tồn tại (404)
> - Tenant isolation: user org A không chuyển được candidate org B (404)
> - Audit log entry được tạo
>
> Sau khi viết test, chạy: pytest tests/test_candidate_move.py -v
```

Sau khi test pass, chạy security review tự động:

```
> /security-review candidates_routes.py

# Claude sẽ tự audit code theo 12 categories trong slash command
# Output: list các issues phát hiện được + đề xuất fix
```

Nếu có CRITICAL/HIGH issues, fix luôn rồi chạy lại security review. KHÔNG commit code có issues open.

Bước 5 — COMMIT

```
> /review

# Claude review toàn bộ thay đổi theo 7 tiêu chí HPU
# Output: pass/fail từng tiêu chí + suggestions

# Sau khi /review clean:
> Commit theo Conventional Commits với message:
> "feat(candidates): add status transition endpoint with 7-state pipeline"
>
> Chia thành 2 commits nếu cần:
> - feat(candidates): add move endpoint
> - test(candidates): add status transition tests
```

Bước 6 — PR

```
> Push branch và tạo PR với template:
> - Title: feat(candidates): add status transition endpoint
> - Description: link tới PLAN.md story BE-007
> - Checklist: tick các mục đã pass
> - Mention @trung-tt cho review
```

Trung review trên Claude.ai web (Workflow B):

PROMPT — Trung review PR (Claude.ai)

Đây là PR từ team:

[paste link GitHub PR + paste code changes]

Review giúp tôi theo 7 tiêu chí HPU + đặc biệt focus:

1. Tenant isolation: có leak không?
2. State transition logic: có cover đủ edge case không?
3. Audit log: có log đúng action không?

Output: list các BLOCKER, HIGH, MEDIUM issues nếu có. Nếu OK, nói rõ "Approved".

3.2. Cách paste code đúng cách (cho Workflow B)

Khi dùng Claude.ai chat, code Claude sinh ra hoặc review chỉ tốt bằng context anh đưa vào. Paste sai cách = output sẽ tệ.

Quy tắc 1 — Luôn báo file path

XẤU:

```
def calculate_score(candidate):
    # ...
```

TỐT:

```
# File: backend/app/services/candidate_service.py

async def calculate_score(candidate: Candidate) -> float:
    # ...
```

Quy tắc 2 — Paste error trace ĐẦY ĐỦ

Khi báo lỗi, paste full traceback từ đầu đến cuối, kèm command đã chạy. Đừng paste mỗi message lỗi cuối.

Quy tắc 3 — Khi paste DB schema, paste cả enum và constraints

```
-- File: schema for candidates table

CREATE TYPE candidate_status AS ENUM (
    'NEW', 'SCREENING', 'TEST_SENT', 'TEST_DONE',
    'INTERVIEW', 'DECISION', 'HIRED', 'REJECTED'
);

CREATE TABLE candidates (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id UUID NOT NULL REFERENCES organizations(id),
    status candidate_status DEFAULT 'NEW',
    -- ... các field khác
    CONSTRAINT uq_candidates_org_email UNIQUE(organization_id, email)
);
```

Lý do: nếu chỉ paste tên cột, Claude không biết constraint, không biết enum value. Sinh code có thể vi phạm constraint.

4. CÁCH DÙNG 7 SKILLS HIỆU QUẢ

Skills là gì?

Skills là các file SKILL.md trong ~/.claude/skills/ với metadata YAML mô tả khi nào skill được trigger.

Claude Code TỰ ĐỘNG load skill khi prompt của anh chứa keywords trong description của skill.

Anh KHÔNG cần gọi skill manually — chỉ cần dùng đúng từ ngữ trong prompt.

4.1. Cách skills được trigger

Mỗi skill có YAML frontmatter:

```
---
name: multi-tenant-saas
description: >
  Multi-tenant SaaS pattern cho FastAPI + PostgreSQL.
  Use when: tenant, multi-tenant, SaaS, organization, isolation,
  RLS, row level security, JWT, subdomain, tenant context,
  cross-tenant, organization_id, TalentChart.
---
```

Khi anh prompt chứa các keyword này, Claude tự load skill content vào context. Tip: dùng từ ngữ tự nhiên, không cần memorize keywords.

4.2. Bảng 7 skills + cách trigger

Skill	Trigger nếu prompt chứa	Khi nào dùng
multi-tenant-saas	tenant, organization, RLS, multi-tenant, isolation	Mọi feature SaaS có data scoping
epa-engine	bát tự, EPA, archetype, ngũ hành, lunar, can chi, tam hợp	Build/sửa logic EPA
nextjs-admin	Next.js, frontend, App Router, shadcn, TanStack, server component	Build UI admin
fastapi-crud	tạo module, CRUD, API endpoint, REST, danh sách, thêm sửa xóa	Build resource mới
docker-deploy	Dockerfile, deploy, container, docker compose, caddy, reverse proxy	Setup/sửa deploy
hpu-ui	Jinja2, Tailwind CDN, sidebar,	Build UI internal tool

Skill	Trigger nếu prompt chứa	Khi nào dùng
	template, dashboard (cho internal tools)	
vietnam-payroll	lương, BHXH, thuế TNCN, biểu thuế, lương net, bảo hiểm	Tính lương VN

4.3. Ví dụ thực tế: skill auto-load

Ví dụ 1 — multi-tenant-saas + fastapi-crud cùng load

PROMPT — Prompt thực tế (TalentChart)

Build module CRUD cho `job_posts` trong TalentChart.

Yêu cầu:

- Multi-tenant: chỉ list jobs của org hiện tại
- 6 endpoints chuẩn (list, get, create, update, delete, export)
- Public endpoint cho candidate xem job đã publish (không auth, dùng subdomain)
- Slug unique trong org

Claude Code sẽ phát hiện 'multi-tenant', 'tenant', 'org' → load skill multi-tenant-saas. Phát hiện 'CRUD', '6 endpoints', 'list/get/create' → load fastapi-crud. Output sẽ tuân thủ cả 2 patterns.

Ví dụ 2 — epa-engine load khi port logic

PROMPT — Port lunar.py từ Fortune HR

Tôi cần port file `lunar.js` của Fortune HR sang Python.

[paste code JS]

Port nguyên xi sang Python theo pattern của TalentChart, KHÔNG cải tiến thuật toán. Test cases verify output match với JS gốc.

Keywords 'port', 'lunar', 'Fortune HR', 'bất tự' → trigger skill epa-engine. Claude sẽ áp dụng đúng pattern: TypedDict, type hints, keep LunarData nguyên xi, viết test verify.

4.4. Khi skill KHÔNG load tự động

Đôi khi anh muốn skill load mà keywords không match. Có 3 cách force load:

Cách 1 — Mention skill name trực tiếp

PROMPT — Force load skill

Đọc skill `~/\.claude/skills/multi-tenant-saas/SKILL.md`.

Sau đó áp dụng pattern đó cho task: ...

Cách 2 — Mention file paths cụ thể

PROMPT — Reference file path

Đọc skill multi-tenant-saas. Implement endpoint candidate detail theo đúng pattern "GET ONE (verify tenant)" trong skill đó.

Cách 3 — Update skill description

Nếu thấy 1 use case lặp đi lặp lại nhưng skill không trigger, update file SKILL.md thêm keywords vào description. Đây là cách cải tiến hpu-dev qua thời gian.

4.5. Khi nào dùng skill nào — quick reference

Cheat sheet

- Build CRUD module mới? fastapi-crud (auto load nếu prompt có 'CRUD', 'module')
- Có dính tenant? Thêm multi-tenant-saas (load nếu có 'tenant', 'org')
- UI admin Next.js? nextjs-admin (load nếu có 'Next.js', 'admin', 'page')
- UI internal Jinja2? hpu-ui (load nếu có 'sidebar', 'template')
- Tính lương? vietnam-payroll (load nếu có 'lương', 'BHXH')
- Port từ Fortune HR? epa-engine (load nếu có 'lunar', 'bát tự', 'port')
- Deploy? docker-deploy (load nếu có 'Docker', 'deploy')

Tip: prompt tự nhiên, đừng cố nhớ keywords. Skills sẽ tự load.

5. CÁCH DÙNG 7 SLASH COMMANDS

Slash commands là gì?

Slash commands là các template prompt định sẵn cho task lặp đi lặp lại.

Trong Claude Code, gõ `/command-name [args]` để invoke.

Đỡ phải gõ lại 1 prompt dài cho task tương tự.

5.1. Bảng 7 slash commands

Command	Args	Mô tả
<code>/new-module</code>	<code>{entity}</code>	Scaffold CRUD module mới (auto detect internal/saas)
<code>/new-tenant</code>	<code>{slug}</code>	Tạo organization mới (chỉ SaaS)
<code>/epa-port</code>	<code>{module}</code>	Port logic JS → Python (chỉ TalentChart)
<code>/security-review</code>	<code>[file/module]</code>	Audit security 12 categories
<code>/review</code>	—	Code review 7 tiêu chí HPU
<code>/deploy</code>	—	Build + deploy Docker
<code>/add-export</code>	<code>{entity}</code>	Thêm xuất Excel cho module

5.2. `/new-module` — dùng nhiều nhất

Ví dụ thực tế

```
# Trong Claude Code (project TalentChart)
> /new-module job_posts

# Claude sẽ:
# 1. Detect project type (SaaS qua CLAUDE.md)
# 2. Hỏi info về fields, permissions, validations
# 3. Scaffold:
#   - src/models/job_post.py (inherit TenantScopedBase)
#   - src/schemas/job_post.py (KHÔNG có org_id)
#   - src/routers/job_post_routes.py (6 endpoints + tenant verify)
#   - src/services/job_post_service.py
#   - alembic/versions/xxx_add_job_posts.py
#   - tests/test_job_posts.py (incl. tenant isolation)
# 4. Register router trong main.py
# 5. Suggest chạy migration
```

Sau khi /new-module xong

- Review code generated, KHÔNG accept mù
- Adjust fields/validation theo nghiệp vụ thực
- Add UI: Next.js page hoặc Jinja2 template
- Add to sidebar nav
- Run migration: docker compose exec backend uv run alembic upgrade head
- Run tests

5.3. /new-tenant — onboard customer mới

```
# Khi có trường ĐH mới đăng ký dùng TalentChart
> /new-tenant truong-x

# Claude sẽ hỏi:
# - Tên đầy đủ tổ chức
# - Tên ngắn
# - Loại (university/college/...)
# - Tier (starter/standard/premier)
# - Email contact
# - Subdomain (default = slug)

# Output:
# - Migration tạo organization row
# - User admin đầu tiên với password mặc định Hpu@2026 (force change)
# - URL admin + public career page
# - Hướng dẫn next steps cho customer
```

5.4. /epa-port — chuyên cho TalentChart

```
# Port logic bắt từ Fortune HR (Express.js) sang Python
> /epa-port lunar

# Claude sẽ:
# 1. Đọc legacy/fortune-hr/backend/server.js (đã extract từ Fortune HR)
# 2. Tạo src/services/epa/lunar.py
# 3. Port nguyên xi LunarData (200 entries) + functions
# 4. Tạo tests/services/epa/test_lunar.py với 20+ test cases
# 5. Test bug fix Lunar Year cho người sinh trước Tết
```

Workflow port EPA Engine đầy đủ

/epa-port lunar → File 1: Lunar conversion 200 năm
 /epa-port canchi → File 2: Thiên Can, Địa Chi, Nạp Âm
 /epa-port menh → File 3: Mệnh ngũ hành
 /epa-port tamhop → File 4: Tam hợp, Xung
 /epa-port compatibility → File 5: Tương hợp 2 người
 /epa-port team-suggest → File 6: Đề xuất đội nhóm

Sau đó tự build src/services/epa/archetype.py (12 archetype fusion — phần MỐI, không port).

Effort: 1-2 ngày cho mid dev với /epa-port. Không có command này: 1 tuần.

5.5. /security-review — chạy trước mọi commit

```

# Audit toàn project
> /security-review

# Audit module cụ thể
> /security-review candidates

# Audit PR hiện tại
> /security-review --pr
  
```

Claude review code theo 12 categories: tenant safety, SQL injection, XSS, auth, authorization, mass assignment, info disclosure, CSRF, crypto, rate limit, dependency vuln, AI anti-patterns.

Output report có format chuẩn: severity (CRITICAL/HIGH/MEDIUM/LOW), location, exploit explanation, fix code patch, test case.

Workflow tích hợp /security-review vào dev flow

1. Sau mỗi feature: chạy /security-review {module}
2. Fix CRITICAL + HIGH issues ngay
3. MEDIUM/LOW: tạo issue GitHub để fix sau
4. Trước deploy production: /security-review (toàn project)

5.6. /review — code review tự động

```
# Trước khi tạo PR, chạy:
> /review

# Claude check 7 tiêu chí HPU:
# 1. API Envelope
# 2. Database Convention
# 3. Validation
# 4. Multi-tenant Safety (SaaS)
# 5. Security
# 6. Code Quality
# 7. UI/UX (nếu có thay đổi UI)
```

Output: pass/fail từng tiêu chí + danh sách BLOCKER/HIGH/MEDIUM issues. Coi như là pre-review trước khi Trung review trên Claude.ai web.

5.7. /deploy — automated deployment

```
# Trên local, sau khi PR merge vào main
> /deploy

# Claude sẽ:
# 1. Pre-flight checks: docker, .env, network
# 2. Run tests + lint
# 3. Build Docker images
# 4. Test locally
# 5. Hỏi user OK push lên server
# 6. SSH lên server, git pull, docker compose up -d
# 7. Verify production health
# 8. Send notification (Telegram nếu config)
```

5.8. /add-export — thêm Excel export

```
# Module candidates đã có CRUD, giờ cần xuất Excel
> /add-export candidates

# Claude sẽ:
# 1. Tạo endpoint GET /api/v1/candidates/export
# 2. Dùng openpyxl với format chuẩn HPU
# 3. Money columns: number_format='#,##0 "đ"'
# 4. Date columns: format='dd/mm/yyyy'
# 5. Header bold, freeze pane
# 6. Filename: candidates-{date}.xlsx
# 7. Permissions: theo permission của list endpoint
```

5.9. Custom slash commands

Khi anh có 1 task lặp đi lặp lại mà chưa có command, tạo file mới trong `.claude/commands/` của project (không phải global, vì command thường specific cho project).

```
# Ví dụ: tạo /add-audit-log để wire audit cho 1 module

# File: .claude/commands/add-audit-log.md
# /add-audit-log – Thêm audit logging cho module
#
# Args: {entity}
#
# Việc cần làm:
# 1. Add audit_log entry trong service create/update/delete
# 2. Verify model AuditLog đã có
# 3. Format: action, resource_type, resource_id, user_id, changes JSON
# 4. Cho mỗi action, tạo audit entry trong cùng transaction
```

6. BỘ PROMPT TEMPLATES ĐÃ TỐI ƯU

Đây là tài sản quan trọng nhất của playbook

Mỗi template đã được test và tinh chỉnh. Copy-paste thẳng vào Claude Code (hoặc Claude.ai chat), thay [biến] cho task của anh.

6.1. Template — Onboarding Claude Code lần đầu

PROMPT — Lần đầu mở Claude Code trên project mới

Đọc kỹ 4 file sau và tóm tắt cho tôi:

1. CLAUDE.md (project instructions)
2. _shared/api-conventions.md (đường dẫn ~/hpu-dev/_shared/)
3. _shared/db-conventions.md
4. docs/PLAN.md (sprint hiện tại nếu có)

Sau đó trả lời:

- Project này làm gì?
- Loại app? (internal hay SaaS?)
- Stack chính?
- Convention quan trọng nhất tôi phải nhớ?
- Skills nào liên quan project?
- Slash commands available?
- Sprint hiện tại đang làm gì?

ĐỪNG sinh code. Đây chỉ là context check.

6.2. Template — Build router CRUD (SaaS)

PROMPT — Build SaaS CRUD router

Tôi cần build router CRUD cho resource [TÊN_RESOURCE] trong TalentChart.

Database table:

[Paste CREATE TABLE statement đầy đủ với enum, constraints, indexes]

Liên kết với:

- [model_a]: [relationship type, ví dụ: many-to-one]
- [model_b]: ...

Endpoints cần build:

- | | | |
|--------|--------------------------|---|
| - GET | /api/v1/[resources] | - List paginated, filter [field1, field2] |
| - POST | /api/v1/[resources] | - Create (auto set org_id) |
| - GET | /api/v1/[resources]/{id} | - Get detail (verify tenant) |


```

- PATCH /api/v1/[resources]/{id}          - Update partial
- DELETE /api/v1/[resources]/{id}         - Soft delete

# Permissions:
- List/Get: HR_STAFF+
- Create/Update: HR_MANAGER+
- Delete: ADMIN+

# Yêu cầu BẮT BUỘC:
1. Tuân thủ skill multi-tenant-saas + fastapi-crud
2. Pydantic schemas: Create (KHÔNG có org_id), Update (all optional), Response (KHÔNG có org_id)
3. Service layer riêng nếu logic >20 dòng
4. Custom exceptions (ResourceNotFound, DuplicateResource)
5. Audit log cho create/update/delete
6. Pagination chuẩn

Trước khi viết code, đề xuất PLAN:
- Cấu trúc các file sẽ tạo/sửa
- Pydantic schemas (field nào required/optional)
- Validation logic đặc biệt
- Edge case cần xử lý
- Test cases cần viết

Tôi review plan rồi confirm.
```

6.3. Template — Build service / business logic

PROMPT — Build service layer

Tôi cần viết service function cho business logic [MÔ_TẢ_NGẮN].

```

# Context:
- Module: [TÊN_MODULE]
- Models liên quan: [list models]

# Business rule chi tiết:
[Mô tả từng bước nghiệp vụ. Ví dụ:]
1. Nhận candidate_id và new_status
2. Verify candidate thuộc organization hiện tại
3. Verify state transition hợp lệ theo pipeline 7 trạng thái:
  - NEW → SCREENING, REJECTED
  - SCREENING → TEST_SENT, REJECTED
  - TEST_SENT → TEST_DONE, REJECTED
  - TEST_DONE → INTERVIEW, REJECTED
  - INTERVIEW → DECISION, REJECTED
```

- DECISION → HIRED, REJECTED

4. Update candidate.status, set updated_at = now()
5. Tạo entry trong status_history table
6. Nếu chuyển sang TEST_SENT: trigger send_test_link_email task (Celery)
7. Audit log entry

Yêu cầu:

- Function signature: `async def move_candidate(session, candidate_id, new_status, current_user) -> CandidateResponse`
- Nếu transition không hợp lệ: `raise InvalidStateTransition` (custom exception)
- Nếu candidate không thuộc org: `raise ResourceNotFound` (KHÔNG `TenantAccessDenied` – tránh leak existence)
- Type hints + docstring
- Test transactional: nếu Celery task fail không được rollback DB

Đề xuất implementation plan trước khi code.

6.4. Template — Refactor multi-tenant (CRITICAL)

PROMPT — Refactor 1 router/service sang multi-tenant

Đây là code đang ở trạng thái SINGLE-TENANT (giả định 1 organization duy nhất). Tôi cần refactor sang MULTI-TENANT.

Code hiện tại:
[Paste full file]

Yêu cầu refactor:

1. Mọi query database PHẢI filter theo organization_id
2. organization_id lấy từ tenant context (đã có sẵn trong `app/core/tenant_context.py`)
3. Mọi response KHÔNG được leak tên của data thuộc org khác
 - Nếu user query candidate id của org B → trả 404 (KHÔNG trả 403)
4. Mọi insert PHẢI tự động set organization_id từ context
5. Validate: trước khi update/delete, verify resource thuộc đúng tenant

Sau khi refactor, anh GHI RÕ:

- Các query đã thay đổi (before/after)
- Nếu có query nào không thay đổi được mà cần raw SQL – DỪNG LẠI và hỏi tôi
- Test cases cần thêm để verify tenant isolation

Anti-patterns CẦN TRÁNH:

- ✗ `session.query(Candidate).filter(id=x).first()` – thiếu org filter
- ✗ Trả về thông tin về resource thuộc org khác (kể cả "không tồn tại trong tổ chức của bạn")
- ✗ Filter ở Python sau khi đã query – phải filter ở SQL

6.5. Template — Port code từ Fortune HR (Express/JS) sang FastAPI/Python

PROMPT — Port logic JS → Python

Tôi cần port chính xác logic này từ Fortune HR (Express.js + better-sqlite3) sang TalentChart (FastAPI + Python + SQLAlchemy).

Code JS gốc (Fortune HR):

[Paste full code JS]

Yêu cầu port:

1. KHÔNG cải tiến thuật toán – port nguyên xi vì code JS đã chạy production ổn định
2. Đổi naming sang snake_case Python
3. Type hints đầy đủ (TypedDict cho dict trả về)
4. Async nếu có I/O, sync nếu pure computation
5. Docstring giải thích thuật toán
6. Comments tiếng Việt cho các phần liên quan văn hoá Á Đông (bát tự, ngũ hành...)

Vị trí file mới:

backend/app/services/epa/[tên_file].py

Yêu cầu test:

Sau khi port, viết test cases SO SÁNH output Python với output JS gốc cho 10-20 input mẫu.

Nếu khác output → có bug ở port, KHÔNG ở code gốc.

Đề xuất plan trước khi code:

1. List functions sẽ port (theo thứ tự dependency)
2. Constants (mảng/dict) cần copy nguyên xi
3. Test cases với input/expected output từ Fortune HR
4. Edge cases (year 1900, year 2100, tháng nhuận, sinh trước Tết)

6.6. Template — Viết test

PROMPT — Viết pytest cho endpoint

Viết test pytest cho endpoint [METHOD] [URL].

Code endpoint:

[Paste router file]

Code service:

[Paste service file]

Pydantic schemas:

[Paste schemas]

```
# Test fixtures đã có sẵn trong conftest.py:
- async_client: AsyncClient để gọi API
- test_org: 1 organization test
- test_org_b: 1 organization khác (để test tenant isolation)
- hr_manager_user: user role hr_manager của test_org
- hr_staff_user: user role hr_staff của test_org
- viewer_user: user role viewer của test_org
- auth_headers(user): trả về dict header với JWT token

# Test cases cần cover:
1. Happy path: hr_manager call API thành công, response shape đúng
2. Validation: input thiếu field required → 422
3. Validation: input sai format → 422
4. Permission: viewer_user call → 403
5. Not found: id không tồn tại → 404
6. Tenant isolation: hr_manager của org_a call với id resource của org_b → 404
7. Audit log: verify entry được tạo
8. Edge case: [tùy task]

# Style:
- Tên test: test_<endpoint>_<scenario> (snake_case)
- Mỗi test 1 assertion chính
- Comments tiếng Việt giải thích test phức tạp
- Dùng pytest.mark.asyncio cho async tests

File test: tests/test_[module].py
```

6.7. Template — Debug lỗi

PROMPT — Debug error

Tôi đang gặp lỗi này, anh giúp tôi debug.

Command đã chạy:

[Ví dụ: docker compose exec backend uv run pytest tests/test_X.py::test_Y -v]

Output đầy đủ (paste FULL traceback):

[Paste]

Code liên quan:

[Paste code endpoint/service đang test]

Code test:

[Paste test]

```
# Tôi đã thử:
1. [Cái gì đã thử]
2. [Cái gì đã thử]

# Phân tích giúp:
1. Root cause là gì? (KHÔNG phải symptom – cái GỐC RỄ)
2. Tại sao bug này tồn tại? (lỗi design, lỗi typo, lỗi understanding?)
3. Đề xuất fix
4. Có cần thêm test case nào để bug này không tái phát?
5. Có chỗ nào tương tự trong codebase có thể có bug giống vậy không?
```

6.8. Template — Architecture decision

PROMPT — Brainstorm architecture decision

Tôi cần ra quyết định kiến trúc cho TalentChart:

Vấn đề:

[Mô tả vấn đề cần ra quyết định. Ví dụ: nên dùng PostgreSQL pgvector hay external Pinecone cho semantic search CV?]

Constraints:

- Budget: ...
- Team capability: 1 mid + 2 intern + Trung
- Phase hiện tại: ...
- Scale dự kiến: ...

Yêu cầu:

Đóng vai SOLUTION ARCHITECT có 10 năm kinh nghiệm SaaS B2B Việt Nam.

Đề xuất:

1. 2-3 options khả thi
2. Cho mỗi option: pros, cons, effort, risks
3. Recommendation của anh + lý do
4. Các yếu tố cần xem xét thêm (anh chưa đủ context)

Output dạng markdown để tôi paste vào docs/DECISIONS.md.

KHÔNG viết code – chỉ thiết kế.

6.9. Template — UI/Frontend Next.js

PROMPT — Build Next.js page

Build trang [TÊN_TRANG] cho TalentChart admin app.

```
# Vị trí:
src/app/(admin)/[route]/page.tsx

# Yêu cầu UI:
[Mô tả UI: list view, detail view, form, dashboard, ...]

# Data:
- API endpoint: [GET/POST] [URL]
- Response shape: { status, data: [...], meta: {...} }
- Pydantic schema (backend): [paste]

# Components cần dùng:
- shadcn/ui: Table, Button, Input, Dialog (đã copy vào src/components/ui/)
- Custom: StatusBadge, formatVND from utils
- TanStack Query hook: use[Resource]s từ hooks/

# Yêu cầu kỹ thuật:
1. Tuân thủ skill nextjs-admin
2. Server Component cho fetch data + SEO
3. Client Component (use client) chỉ cho interactive parts
4. Loading state qua loading.tsx
5. Error state qua error.tsx
6. Form: React Hook Form + Zod validation
7. Mobile responsive (md:grid-cols-2 → 1 col mobile)
8. HPU palette: bg-hpu-primary cho header, sidebar đã có ở layout

Đề xuất plan trước khi code:
- Files sẽ tạo (page.tsx, loading.tsx, error.tsx, components nào)
- Component breakdown
- Data flow (server fetch hoặc client TanStack Query?)
```

7. CODE REVIEW CHECKLIST

Mọi PR phải pass checklist này trước khi merge

Trung là người duy nhất merge vào main. Trước khi anh approve PR, copy checklist này, tick từng box. Một box nào không tick được → request changes.

7.1. Multi-tenant safety (CRITICAL — fail = block merge)

- ☐ Mọi query trên TenantScopedBase model có filter organization_id (auto qua middleware hoặc explicit)
- ☐ Không có raw SQL — chỉ SQLAlchemy ORM
- ☐ Endpoint nhận id của resource: code có verify resource thuộc đúng tenant trước khi process
- ☐ Endpoint trả 404 (không phải 403) khi resource không thuộc tenant — tránh leak existence
- ☐ Test có ít nhất 1 case 'tenant isolation': user của org A không truy cập được resource của org B
- ☐ Pydantic schema KHÔNG cho phép set organization_id từ client
- ☐ Pydantic Response schema KHÔNG include organization_id

7.2. Authentication & Authorization

- ☐ Endpoint protected có dependency Depends(get_current_user)
- ☐ Permission check theo đúng spec (org_role được phép)
- ☐ Token validate đúng: signature, expiration, organization_id match
- ☐ Public endpoint (login, careers public) không có dependency auth, có rate limit
- ☐ Password không log, không trả về response
- ☐ JWT secret không có default value trong code

7.3. Input validation

- ☐ Request body validate qua Pydantic schema (không validate tay)
- ☐ Query params có type hints rõ ràng
- ☐ String fields có max_length
- ☐ Numeric fields có min/max constraints
- ☐ Email fields dùng EmailStr
- ☐ Enum fields dùng đúng Python Enum (không string thuần)

7.4. Error handling

- ☐ Custom exception cho domain errors (ResourceNotFound, ValidationError, ...)
- ☐ HTTP status code đúng cho mỗi exception (404, 422, 403, ...)

- ☐ Error message tiếng Việt cho user, tiếng Anh trong log
- ☐ Không leak stack trace ra response production
- ☐ Không catch generic Exception và swallow

7.5. Database

- ☐ Mọi query dùng async/await
- ☐ Session.commit() có try/except để rollback nếu fail
- ☐ N+1 query: dùng selectinload() hoặc joinedload() cho relationships
- ☐ Index có cho các cột thường query
- ☐ Migration có cả upgrade() và downgrade()
- ☐ Soft delete (status='inactive'), không db.delete()

7.6. API design

- ☐ URL theo REST conventions (/resources, /resources/{id})
- ☐ HTTP method đúng (GET đọc, POST tạo, PATCH sửa, DELETE xóa)
- ☐ Response shape consistent: {status, data, message} qua helper
- ☐ Pagination params chuẩn: ?page, ?per_page
- ☐ Filter/sort params trong query string

7.7. Logging & Observability

- ☐ Mọi endpoint có log start/end với context
- ☐ Mọi exception ngoài 4xx có log với traceback
- ☐ Background task có log start/end/error
- ☐ Không log password, token, full PII
- ☐ Audit log entry cho action sửa data quan trọng

7.8. Testing

- ☐ Có ít nhất 1 happy path test
- ☐ Có test cho permission denied
- ☐ Có test cho tenant isolation (SaaS)
- ☐ Có test cho validation errors (422)
- ☐ Có test cho not found (404)
- ☐ Test pass: pytest -v
- ☐ Coverage không giảm

7.9. Code quality

- ☐ Type hints đầy đủ cho function signatures
- ☐ Docstring cho function public
- ☐ Tên biến/hàm rõ nghĩa
- ☐ File <500 dòng
- ☐ Function <50 dòng
- ☐ ruff check pass
- ☐ mypy pass
- ☐ Không có TODO/FIXME/your-key-here sót

7.10. UI/UX (nếu có thay đổi UI)

- ☐ Dùng palette HPU (#1e3a5f)
- ☐ Sidebar 240px, layout đồng nhất
- ☐ Money format N.NNN.NNN đ
- ☐ Date display DD/MM/YYYY
- ☐ Error message tiếng Việt
- ☐ Loading + Error states cho mỗi page (Next.js)
- ☐ Mobile responsive

8. 12 ANTI-PATTERNS AI HAY SINH RA

Đây là 12 lỗi phổ biến nhất khi vibe coding với Claude/Cursor

Đọc kỹ. Sau mỗi PR, tự kiểm xem có vướng anti-pattern nào không. /security-review sẽ bắt được phần lớn nhưng không phải tất cả.

Anti-pattern 1 — Filter ở Python sau khi đã query

Triệu chứng:

```
# WRONG – query toàn bộ rồi filter Python
async def list_candidates(session, current_user):
    all_candidates = await session.execute(select(Candidate))
    filtered = [c for c in all_candidates
                if c.organization_id == current_user.organization_id]
    return filtered
```

Vấn đề: leak data của org khác qua memory + chậm khủng khiếp khi 10K+ records.

Đúng:

```
# CORRECT – filter ở SQL với org context
async def list_candidates(session, current_user):
    result = await session.execute(
        select(Candidate)
        .where(Candidate.organization_id == current_user.organization_id)
    )
    return result.scalars().all()
```

Anti-pattern 2 — Trả model trực tiếp qua API

```
# WRONG
@router.get("/users/{id}")
async def get_user(id: UUID, session = Depends(get_session)):
    user = await session.get(User, id)
    return user # Leak hashed_password, internal fields, organization_id
```

Đúng: tạo Pydantic Response schema riêng, chỉ chứa fields được phép expose.

```
@router.get("/users/{id}", response_model=UserResponse)
async def get_user(id: UUID, session = Depends(get_session), user =
Depends(get_current_user)):
```

```

org_id = get_current_org_id_required()
result = await session.execute(
    select(User).where(User.id == id).where(User.organization_id == org_id)
)
user = result.scalar_one_or_none()
if not user:
    raise ResourceNotFound("User")
return success(UserResponse.model_validate(user))

```

Anti-pattern 3 — Default fallback cho secrets

```

# WRONG – SmartHire hiện tại có pattern này, ĐỪNG copy
JWT_SECRET = os.getenv("JWT_SECRET", "change-me-in-production")

```

Vấn đề: nếu deploy quên set env, secret default sẽ leak — anyone có thể forge JWT.

```

# CORRECT – fail fast
class Settings(BaseSettings):
    JWT_SECRET: str # Pydantic raise ValidationError nếu thiếu
    ANTHROPIC_API_KEY: str
    DATABASE_URL: str

    model_config = SettingsConfigDict(env_file=".env")

settings = Settings() # Fail at import time if missing env

```

Anti-pattern 4 — N+1 query

```

# WRONG
candidates = await session.execute(select(Candidate))
for candidate in candidates.scalars():
    print(candidate.organization.name) # Trigger N queries!

```

```

# CORRECT – eager load với selectinload
candidates = await session.execute(
    select(Candidate)
    .options(selectinload(Candidate.organization))
)
for candidate in candidates.scalars():
    print(candidate.organization.name) # Single query

```

Anti-pattern 5 — Catch và swallow exception

```
# WRONG
try:
    result = await some_service()
except Exception:
    pass # Silent fail, user không biết, dev không biết
```

```
# CORRECT – log đầy đủ + raise lại
try:
    result = await some_service()
except DomainError as e:
    logger.error("Domain error in some_service", error=str(e))
    raise HTTPException(400, str(e))
except Exception as e:
    logger.exception("Unexpected error in some_service")
    raise HTTPException(500, "Lỗi hệ thống")
```

Anti-pattern 6 — Time.sleep() trong async

```
# WRONG – block event loop, kill performance
async def send_email(to: str):
    time.sleep(2) # Block toàn bộ server cho mọi request!
    smtp.send(...)

# CORRECT – async sleep
async def send_email(to: str):
    await asyncio.sleep(2) # Yield control
    await aiosmtp.send(...)
```

Anti-pattern 7 — Mass assignment qua Pydantic

```
# WRONG – UserCreate có field organization_id, role
class UserCreate(BaseModel):
    username: str
    email: str
    role: str
    organization_id: UUID # Client có thể tự set!

@router.post("/users")
async def create_user(data: UserCreate, ...):
    user = User(**data.model_dump()) # Client kiểm soát role + org!
```

```
# CORRECT – schema riêng, set internal field từ context
class UserCreate(BaseModel):
    username: str
    email: EmailStr
    full_name: str
    # KHÔNG có role, organization_id

@router.post("/users")
async def create_user(
    data: UserCreate,
    current_user = Depends(require_org_role(OrgRole.ADMIN))
):
    user = User(
        **data.model_dump(),
        organization_id=current_user.organization_id, # From context
        org_role="member" # Default
    )
```

Anti-pattern 8 — IDOR (Insecure Direct Object Reference)

```
# WRONG – không verify ownership
@router.get("/candidates/{id}")
async def get_candidate(id: UUID, session = Depends(get_session)):
    candidate = await session.get(Candidate, id)
    return candidate # User của org A có thể xem candidate của org B!
```

```
# CORRECT
@router.get("/candidates/{id}")
async def get_candidate(
    id: UUID,
    session = Depends(get_session),
    current_user = Depends(get_current_user)
):
    org_id = get_current_org_id_required()
    result = await session.execute(
        select(Candidate)
        .where(Candidate.id == id)
        .where(Candidate.organization_id == org_id)
    )
    candidate = result.scalar_one_or_none()
    if not candidate:
        raise ResourceNotFound("Candidate") # 404, KHÔNG 403
    return success(CandidateResponse.model_validate(candidate))
```

Anti-pattern 9 — Log thông tin nhạy cảm

```
# WRONG
logger.info(f"User logged in: {user.email}, password: {password}")
logger.info(f"JWT token: {token}")
logger.info(f"DB query: {query_with_pii}")

# CORRECT
logger.info("User logged in", user_id=user.id, tenant_id=user.organization_id)
# KHÔNG log password, token, PII
```

Anti-pattern 10 — TODO/FIXME/your-api-key sót lại

AI hay sinh code có placeholder kiểu:

```
# AI thường sinh code có dòng này
api_key = "your-anthropic-api-key-here" # TODO: get from env
candidate.score = 100 # FIXME: calculate properly later
return {"status": "TODO"}
```

Phải grep trước khi commit:

```
# Pre-commit hook tự động grep
git diff --cached | grep -E "TODO|FIXME|your-.*-here|XXX"
# Nếu match, fail commit
```

Anti-pattern 11 — Race condition trong concurrent request

```
# WRONG – race condition
async def increment_view_count(job_id: UUID, session):
    job = await session.get(JobPost, job_id)
    job.view_count += 1 # Read
    await session.commit() # Write
    # Nếu 100 request cùng lúc, view_count chỉ tăng vài lần

# CORRECT – atomic update
from sqlalchemy import update

async def increment_view_count(job_id: UUID, session):
    await session.execute(
        update(JobPost)
        .where(JobPost.id == job_id)
        .values(view_count=JobPost.view_count + 1)
    )
```

```
await session.commit()
```

Anti-pattern 12 — Trust client cho expensive operation

```
# WRONG – client kiểm soát số records query
@router.get("/candidates")
async def list_candidates(
    page_size: int = Query(20),
    session = Depends(get_session)
):
    # Nếu client gửi page_size=999999, server sẽ chết
    result = await session.execute(
        select(Candidate).limit(page_size)
    )

# CORRECT – clamp values
@router.get("/candidates")
async def list_candidates(
    page_size: int = Query(20, ge=1, le=100), # Max 100
    session = Depends(get_session)
):
    result = await session.execute(
        select(Candidate).limit(page_size)
    )
```

9. WORKFLOW GIT VÀ CI/CD

9.1. Git branch strategy

TalentChart Phase 1 dùng GitHub Flow đơn giản:

```
main                ← Production-ready, deploy được tự động
├ feature/BE-001-add-candidate-router
├ feature/FE-002-login-page
├ feature/BE-003-epa-port-lunar
└ fix/BE-007-tenant-leak-fix
```

- main: branch chính, luôn deploy được. Chỉ Trung merge.
- feature/[ticket-id]-[ngắn-mô-tả]: branch dev, mỗi PR 1 branch
- fix/[ticket-id]-....: cho hot fix
- Không có develop branch riêng — Phase 1 không cần

9.2. Commit message convention

```
<type>(<scope>): <subject>

[optional body]

[optional footer: refs/closes #ticket]
```

Type: feat, fix, refactor, docs, test, chore

Ví dụ tốt:

```
feat(epa): port lunar.py from Fortune HR

- Port LunarData array 1900-2100
- Implement get_lunar_date() with same algorithm as JS version
- Add tests with 100 sample dates verified against JS output

Refs: #BE-015

---

fix(candidates): prevent IDOR in get_candidate endpoint

Verify candidate.organization_id matches current user's org
before returning. Return 404 (not 403) to avoid leaking existence.

Closes: #SEC-003
```


9.3. Pull Request template

File `.github/pull_request_template.md` (đã có sẵn trong project sau khi `/new-project.sh`):

```
## Mô tả

[Mô tả ngắn what và why của PR. Link tới ticket]

Refs: #BE-XXX

## Loại thay đổi

- [ ] feat – Feature mới
- [ ] fix – Sửa bug
- [ ] refactor – Refactor không đổi behavior
- [ ] docs – Documentation
- [ ] test – Tests
- [ ] chore – Misc

## Checklist (BẮT BUỘC tick mới được merge)

### Multi-tenant safety (chỉ SaaS)
- [ ] Mọi query có filter organization_id
- [ ] Không có raw SQL
- [ ] Test có case tenant isolation
- [ ] Cross-tenant trả 404 (không 403)

### Security
- [ ] Đã chạy /security-review
- [ ] Không log PII/password/token
- [ ] Không có default secrets
- [ ] Pydantic schema không cho mass assignment

### Code quality
- [ ] /review pass
- [ ] ruff + mypy + tests pass
- [ ] Coverage không giảm
- [ ] Không có TODO/FIXME sót

### Documentation
- [ ] OpenAPI doc tự động generate đúng
- [ ] CHANGELOG.md updated nếu là feature lớn
- [ ] DECISIONS.md updated nếu có ADR mới

## Cách test
```

[Hướng dẫn reviewer]

Screenshots (nếu là frontend)

9.4. CI pipeline (GitHub Actions)

```
# .github/workflows/ci.yml

name: CI

on:
  pull_request:
    branches: [main]

jobs:
  test-backend:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16-alpine
        env:
          POSTGRES_PASSWORD: testpw
          POSTGRES_DB: talentchart_test
        ports: [5432:5432]
      redis:
        image: redis:7-alpine
        ports: [6379:6379]

    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install uv
      - run: cd backend && uv sync --frozen
      - run: cd backend && uv run ruff check .
      - run: cd backend && uv run mypy app/
      - run: cd backend && uv run alembic upgrade head
      env:
        DATABASE_URL_SYNC:
        postgresql+psycpg2://postgres:testpw@localhost/talentchart_test
      - run: cd backend && uv run pytest --cov=app --cov-report=xml
      env:
        DATABASE_URL:
        postgresql+asyncpg://postgres:testpw@localhost/talentchart_test
        REDIS_URL: redis://localhost:6379/0
        JWT_SECRET: test-secret-32-chars-minimum-please
```

```
    ANTHROPIC_API_KEY: dummy-not-called-in-tests
  - uses: codecov/codecov-action@v4

test-frontend:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with: { node-version: "20" }
    - run: cd frontend && pnpm install --frozen-lockfile
    - run: cd frontend && pnpm lint
    - run: cd frontend && pnpm typecheck
    - run: cd frontend && pnpm test
    - run: cd frontend && pnpm build
```

10. KHI NÀO KHÔNG DỪNG AI

AI không phải lúc nào cũng là answer

Có những việc dev tay nhanh hơn, an toàn hơn, hoặc bắt buộc phải dev tay.

10.1. Việc TUYỆT ĐỐI không dùng AI sinh

- Code xử lý tiền/payment — Phase 2 nếu có. AI có thể sai logic, mất tiền thật.
- Cryptographic primitives — đừng để AI viết encryption từ đầu, dùng library chuẩn.
- Database migration cho production data — viết tay, review kỹ, test trên staging.
- Code production secret management — secret manager, KMS, vault.
- Compliance/legal text — Privacy Policy, ToS phải có lawyer review.
- Brand voice content — copywriting cho marketing, AI chỉ làm draft, người sửa cuối.
- EPA Engine: 12 archetype description — đây là core IP của TalentChart, Trung viết riêng, không AI.

10.2. Việc dev tay nhanh hơn AI

- Đổi tên 1 biến, 1 file (rename refactor) — IDE/grep nhanh hơn
- Format code — ruff/prettier auto
- Fix typo — sửa tay 5 giây
- Update version trong package.json/pyproject.toml — text edit
- Commit, push, PR — Git command (Claude Code có thể làm nhưng overhead)

10.3. Việc cần thận trọng với AI

- EPA Engine logic — port nguyên xi, đừng để AI 'cải tiến' thuật toán bất tự
- Multi-tenant filter logic — review từng query
- Auth flow — verify từng dependency, từng decoder
- Migration với data backfill — chạy dry run trước
- Regular expression — AI hay sinh regex sai cho edge case (đặc biệt với Tiếng Việt có dấu)
- Date/timezone handling — Tiếng Việt + lịch âm là combo khó, dễ sai

11. PHÂN CHIA CÔNG VIỆC CHO TEAM 1+2+1

11.1. Cấu trúc team Phase 1

1 + 2 + 1 — đội hình lý tưởng cho 100tr/Phase 1

- 1 mid dev fulltime — tech lead implementation
- 2 sinh viên CNTT HPU intern part-time (15-20h/tuần)
- 1 Trung kiểm PM/Architect/DevOps — không full code, chỉ design + review + merge

11.2. Phân chia trách nhiệm

Người	Trách nhiệm	% công sức Phase 1
Trung	Architecture, Code review, Merge PR, DevOps, ADRs	30%
Mid dev	Backend chính, EPA Engine, Multi-tenant, security	60%
Intern 1	Frontend Next.js, components, public pages	70%
Intern 2	Backend đơn giản: CRUD, schemas, tests	50%

11.3. Phân chia module Phase 1

Module/Component	Owner chính	Hỗ trợ
Multi-tenant infrastructure	Trung + Mid dev	—
Auth, RBAC	Mid dev	Intern 2
Module 1 — Job Board (admin)	Mid dev	Intern 2
Module 1 — Career Page (public)	Intern 1	Mid dev
Module 2 — ATS core	Mid dev	Intern 2
Module 2 — Pipeline UI	Intern 1	—
Module 3 — EPA Engine port	Mid dev	Trung review
Module 3 — 12 Archetype fusion	Mid dev	Trung design
Module 3 — EPA UI	Intern 1	—
Frontend layout, design system	Intern 1	—
Tests (backend)	Intern 2	Mid dev

Module/Component	Owner chính	Hỗ trợ
Tests (frontend E2E)	Intern 1	—
DevOps, Docker, CI	Trung	Mid dev

11.4. Daily routine

Lịch họp hằng ngày + tuần

Daily standup (15 phút, 9:00 sáng):

- Mỗi người: 'Hôm qua làm gì? Hôm nay làm gì? Có blocker không?'
- KHÔNG dài hơn 15 phút. Ai cần discuss sâu hẹn 1-1 sau

Weekly review (1 tiếng, sáng thứ 2):

- Review tiến độ tuần trước
- Plan tuần mới (sprint planning)
- Demo các feature đã hoàn thành

Code review SLA:

- PR mở trước 12h trưa → review trong ngày
- PR mở sau 12h trưa → review sáng hôm sau
- Critical fix → review trong 2 giờ

11.5. Communication channels

- Daily code: Slack/Telegram channel #talentchart-dev
- Code review: GitHub PR comments (không Slack DM về code)
- Architecture decisions: Notion page với ADR
- Bug tracking: GitHub Issues với labels (P0/P1/P2)
- Sprint planning: Linear hoặc GitHub Projects

11.6. Onboarding intern mới

Roadmap onboarding 1 tuần cho intern

Day 1 — Setup môi trường:

- Cài Docker, Node.js, Python
- Cài Claude Code + login
- Giải nén hpu-dev v2.0 + chạy setup.sh
- Clone repo TalentChart, chạy được local

Day 2 — Đọc tài liệu:

- Đọc 3 file: File 1 (Concept), File 2 (Architecture), File 3 (Playbook này)
- Đọc ~/hpu-dev/_shared/api-conventions.md, db-conventions.md
- Đọc CLAUDE.md của TalentChart

Day 3-4 — Task nhỏ với Claude Code:

- Nhận 1 task đơn giản (ví dụ: thêm 1 field cho candidates)
- Làm theo workflow 6 bước
- Chạy /review + /security-review
- Tạo PR đầu tiên

Day 5 — Pair programming với mid dev:

- Mid dev demo cách dùng skills + slash commands
- Cùng làm 1 task vừa
- Học cách handle khi AI sinh code sai

12. ĐO LƯỜNG NĂNG SUẤT + CẢI TIẾN

12.1. Metrics cần track

Đo lường để biết vibe coding có thực sự tăng năng suất không, có chỗ nào đang cản trở không.

Metric	Mục tiêu Phase 1	Cách đo
Tốc độ đóng PR	<3 ngày trung bình	GitHub PR data
Số PR/dev/tuần	5-10 PRs	GitHub PR data
Code coverage	>70%	Codecov tự động
Bug tỷ lệ post-merge	<1 bug/10 PR	GitHub issues sau merge
Lint/type errors	0	CI check
Time spent per task	<2h cho task vừa	Self-report sprint review
AI suggestion acceptance rate	60-80%	Self-report
Security review findings	<2 issues/PR	Tự đếm

12.2. Retrospective hàng sprint

Mỗi sprint (2 tuần) làm 1 retro 30 phút. Hỏi 4 câu:

- Cái gì làm tốt — keep doing
- Cái gì làm chưa tốt — stop doing
- Cái gì cần thử mới — start doing
- Có ai cần help — ask for help

Đặc biệt cho vibe coding, hỏi thêm:

- Prompt template nào hay dùng/đã viết — share với team
- Lỗi AI sinh ra hay gặp — bổ sung vào anti-pattern list
- Có task nào AI làm tệ hơn dev tay — học để dùng đúng tool
- Có cần thêm skill / slash command nào không?

12.3. Cải tiến hpu-dev định kỳ

Bộ hpu-dev là document sống, không phải kinh thánh. Mỗi tháng:

- Review skills — bổ sung pattern mới từ kinh nghiệm thực
- Update slash commands — thêm command cho task lặp đi lặp lại
- Update anti-pattern list — bug nào lặp lại, thêm vào
- Update review checklist — tiêu chí nào không còn cần, bỏ

- Update prompt templates — thêm template mới khi team viết được
- Trung tổng kết, share lại với team, push lên repo hpu-dev shared

hpu-dev là tài sản trí tuệ của HPU

Mỗi cải tiến hpu-dev = tăng năng suất cho mọi project HPU sau này.
Phase 1 TalentChart là thử nghiệm thực tế quy mô lớn nhất của hpu-dev.
Sau Phase 1, bộ hpu-dev v3.0 sẽ là phiên bản đã được battle-tested.

12.4. Lời nhắn cuối

Ghi nhớ

Vibe coding với Claude Code + meta-architecture HPU là superpower
NẾU dev biết cách. Là cách tự sát NẾU dev lười nghĩ.

Làm theo playbook này: anh và team sẽ build TalentChart Phase 1
trong 4 tháng với chất lượng cao hơn nhiều dự án truyền thống 8 tháng.

Với cùng team, cùng budget, làm gấp đôi việc.

Đó là lý do tồn tại của bộ hpu-dev và playbook này.